

Engineering Reboot 2026

Week 02 - Day 07 Field Manual

Boolean Logic

MISSION BRIEF

Today's objective is to extend the conditional logic skills acquired on Day 06 by introducing Boolean operators.

Programs will evaluate multiple conditions simultaneously using:

- and
- or
- not

These operators form the foundation of:

- Software engineering
- Access control systems
- Authentication systems
- Automation systems
- Industrial control systems
- Embedded firmware
- Security systems

ENGINEERING CONTEXT

Building Access System

Employee Badge Valid
AND
Authorized Time Window

=> Unlock Door

Bank Login

Username Correct
AND
Password Correct

=> Grant Access

Fire Alarm

Smoke Detected
OR
Heat Threshold Exceeded

=> Activate Alarm

Safety Interlock

NOT Emergency Stop Activated

=> Permit Operation

LEARNING OBJECTIVES

By the end of Day 07 you should understand:

- True

- False
- Boolean expressions
- AND operator
- OR operator
- NOT operator
- Multi-condition decisions
- Combining comparisons
- Decision trees

DELIVERABLE INVENTORY

Required Deliverables

1. eligibility_checker.py
2. building_access_control.py
3. Notes.md
4. Journal.md
5. Evidence Package

ARTIFACT PLACEMENT MATRIX

Exercises

```

Week-02/
■■■ Exercises/
    ■■■ Day-07/
        ■■■ eligibility_checker.py
        ■■■ building_access_control.py
  
```

Notes

```

Week-02/
■■■ Notes/
    ■■■ Day-07/
        ■■■ Notes.md
  
```

Journal

```

Week-02/
■■■ Journal/
    ■■■ Day-07/
        ■■■ Journal.md
  
```

Evidence

```

Week-02/
■■■ Day-07/
    ■■■ Evidence/
  
```

EXERCISE 1 - ELIGIBILITY CHECKER

Objective

Determine whether a person qualifies for a service.

Filename

eligibility_checker.py

Repository Location

Week-02/Exercises/Day-07/

Input Type

NUMERIC

Input

Age

```
Processing Rule
-----
Eligible when:

Age >= 18
AND
Age <= 65

Outputs
-----
Eligible
Not Eligible

Validation Matrix
-----
17  -> Not Eligible
18  -> Eligible
35  -> Eligible
65  -> Eligible
66  -> Not Eligible
```

EXERCISE 2 - BUILDING ACCESS CONTROL

```
Objective
-----
Simulate a building access authorization system.

Filename
-----
building_access_control.py

Repository Location
-----
Week-02/Exercises/Day-07/

Inputs
-----
Employee Status (Y/N)
Hour Of Day (0-23)

Processing Rule
-----
Employee = Y
AND Hour >= 8
AND Hour <= 17

Outputs
-----
Access Granted
Access Denied

Validation Matrix
-----
Y / 10  -> Access Granted
N / 10  -> Access Denied
Y / 07  -> Access Denied
Y / 18  -> Access Denied
```

EVIDENCE MATRIX

```
eligibility_checker-Code.png
eligibility_checker-Eligible.png
eligibility_checker-NotEligible.png

building_access_control-Code.png
building_access_control-AccessGranted.png
building_access_control-AccessDenied.png

Git-Status.png
Git-Commit.png
Git-Push-Success.png
GitHub-Remote-Verification.png
```

NOTES TEMPLATE

```
# Day 07 Notes
## Concepts Learned
## Key Commands
## Key Observations
## Lessons Learned
## Questions For Future Study
```

JOURNAL TEMPLATE

```
# Day 07 Journal
## Summary
## Challenges Encountered
## Solutions Applied
## Confidence Assessment
## Reflection
## Tomorrow Preparation
```

GIT WORKFLOW

```
git status
git add .
git commit -m "Complete Day 07 boolean logic exercises"
git push
```

REMOTE VERIFICATION

```
git fetch
git status
```

```
git branch -vv
```

Expected Result

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean.

EXPECTED REPOSITORY END STATE

```
Week-02/
■
■■■ Exercises/
■   ■■■ Day-07/
■       ■■■ eligibility_checker.py
■       ■■■ building_access_control.py
■
■■■ Notes/
■   ■■■ Day-07/
■       ■■■ Notes.md
■
■■■ Journal/
■   ■■■ Day-07/
■       ■■■ Journal.md
■
■■■ Day-07/
    ■■■ Evidence/
```

END-OF-DAY SUCCESS CRITERIA

- [] eligibility_checker.py completed
- [] building_access_control.py completed
- [] Validation cases executed
- [] Evidence captured
- [] Notes completed
- [] Journal completed
- [] Git commit completed
- [] Git push completed
- [] Remote synchronization verified
- [] Working tree clean

DAY 08 PREVIEW

Tomorrow's Topic

LOOPS

Deliverables

- counting_program.py
- even_number_generator.py
- sum_calculator.py

Core Concepts

- for loops
- range()
- counting
- repetition
- accumulation